SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : SK MOHAMMED IRFAN

Reg. No: 11249A342

Class : II B.E. (CSE)

Section: S4

Course Code: BCSF244TA12

Course Name: DATA SCIENCE TOOLKIT LAB

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by **Mr./Ms. SK MOHAMMED IRFAN**,

with **Reg. No 11249A342** of **II-B.E.(CSE)** during the academic year 2025 – 2026.

Station: ENATHUR

Date:

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on_____.

Examiner-1 **Examiner-2**

INDEX

SL.NO	Name of the experiment	Date	Page no	Signature
1(a)	NumPy	19/01/2026		
1(b)	Pandas	02/02/2026		
1(c)	Matplotlib	02/02/2026		
1(d)	Scikit	09/02/2026		
2	Perform data exploration and preprocessing in Python	15/02/2026		
3	Implement a regularized linear regression	23/02/2026		
4	Implement Navies bayes classifier	23/02/2026		
5	Implement regularized logistic regression	02/03/2026		
6	Build models using different ensembling techniques	23/03/2026		
7	Build models using decision tree	23/03/2026		
8	Support Vector Machine with different kernels	23/03/2026		
9	Build model using SVM with different kernels	30/03/2026		
10	Build models to perform clustering using K means after applying PCA & determining the value of K using elbow method	30/03/2026		

SL.NO	Name of the experiment	Date	Page no	Signature

1.Introduction to Python Libraries- NumPy, Pandas, Matplotlib, Scikit.

1.1 NUMPY

```
import numpy as np
a = np.array([1,2,3,4,5])
b = np.arange(1,10,2)
c = np.linspace(0,1,5)
d = np.zeros((2,3))
e = np.ones((2,2))
f = np.eye(3)
g = np.random.randint(1,100,size=(3,3))
print("ARRAY :\n",a)
print("ARANGE :\n",b)
print("LINSPACE :\n",c)
print("ZEROS :\n",d)
print("ONES :\n",e)
print("IDENTITY :\n",f)
print("RANDOM :\n",g)
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

Output:

```
ARRAY :
 [1 2 3 4 5]
ARANGE :
 [1 3 5 7 9]
LINSPACE :
 [0.   0.25 0.5  0.75 1.  ]
ZEROS :
 [[0. 0. 0.]
 [0. 0. 0.]]
ONES :
 [[1. 1.]
 [1. 1.]]
IDENTITY :
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
RANDOM :
 [[15 36 87]
 [29 85 63]
 [25 37 99]]
```

```python
print("ADD:\n",np.add(a,2))
print("MULTIPLY:\n",np.multiply(a,3))
print("SQUARE ROOT:\n",np.sqrt(a))
print("POWER:\n",np.power(a,2))
print("ABSOLUTE:\n",np.abs([-1,-2,3]))
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
ADD:
 [3 4 5 6 7]
MULTIPLY:
 [ 3  6  9 12 15]
SQUARE ROOT:
 [1.         1.41421356 1.73205081 2.         2.23606798]
POWER:
 [ 1  4  9 16 25]
ABSOLUTE:
 [1 2 3]
```

```python
print("MEAN:\n",np.mean(a))
print("MEDIAN:\n",np.median(a))
print("STANDARD DEVIATION:\n",np.std(a))
print("VARIANCE:\n",np.var(a))
print("MINIMUM:\n",np.min(a))
print("MAXIMUM:\n",np.max(a))
print("SUM:\n",np.sum(a))
print("PERCENTILE:\n",np.percentile(a,50))
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
MEAN:
 3.0
MEDIAN:
 3.0
STANDARD DEVIATION:
 1.4142135623730951
VARIANCE:
 2.0
MINIMUM:
 1
MAXIMUM:
 5
SUM:
 15
PERCENTILE:
 3.0
```

```python
print("COLUMN-WISE SUM:\n",np.sum(g,axis=0))
print("ROW-WISE MEAN:\n",np.mean(g,axis=1))
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
COLUMN-WISE SUM:
 [ 69 158 249]
ROW-WISE MEAN:
 [46.         59.         53.66666667]
```

```python
print("FIRST ELEMENT:\n",a[0])
print("SLICE:\n",a[1:4])
print("2D SLICE:\n",g[0:2,1:3])
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
FIRST ELEMENT:
 1
SLICE:
 [2 3 4]
2D SLICE:
 [[36 87]
 [85 63]]
```

```python
print("WHERE > 50:\n",np.where(g>50))
print("ANY > 50:\n",np.any(g>90))
print("ALL > 0:\n",np.all(g>0))
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
WHERE > 50:
 (array([0, 1, 1, 2]), array([2, 1, 2, 2]))
ANY > 50:
 True
ALL > 0:
 True
```

```python
print("SORTED a:\n",np.sort(a))
print("INDEX OF MAX:\n",np.argmax(a))
print("INDEX OF MIN:\n",np.argmin(a))
print("UNIQUE ELEMENTS:\n",np.unique([1,2,2,4,4]))
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
SORTED a:
 [1 2 3 4 5]
INDEX OF MAX:
 4
INDEX OF MIN:
 0
UNIQUE ELEMENTS:
 [1 2 4]
```

```python
x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])
print("DOT PRODUCT\n:",np.dot(x,y))
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
DOT PRODUCT
: [[19 22]
 [43 50]]
```

1.2 PANDAS

A) Creating Data Frames

```python
import pandas as pd
data ={"Name":['Alice','Bob','Charlie'],"Age":[25,30,35],"City":['Newyork','Paris','London']}
df=pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

Output:

```
...        Name  Age     City
0    Alice   25  Newyork
1      Bob   30    Paris
2  Charlie   35   London
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
        Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
```

B) Selecting and filtering

```
ages=df['Age']
above_25=df[df['Age']>25]
row_0=df.iloc[0]
specific_val=df.loc[0,'City']
print(above_25)
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
...        Name  Age    City
1        Bob   30   Paris
2    Charlie   35  London
```

C)Data Cleaning (Handling Missing Values)

```
print(df.isnull().sum())
df_clean=df.dropna()
df_filled=df.fillna(0)
df['Age']=df['Age'].fillna(df['Age'].mean())
print(df['Age'])
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
...  Name    0
Age     0
City    0
dtype: int64
0    25
1    30
2    35
Name: Age, dtype: int64
```

1.3) MATPLOTLIB

Basic Plot Program

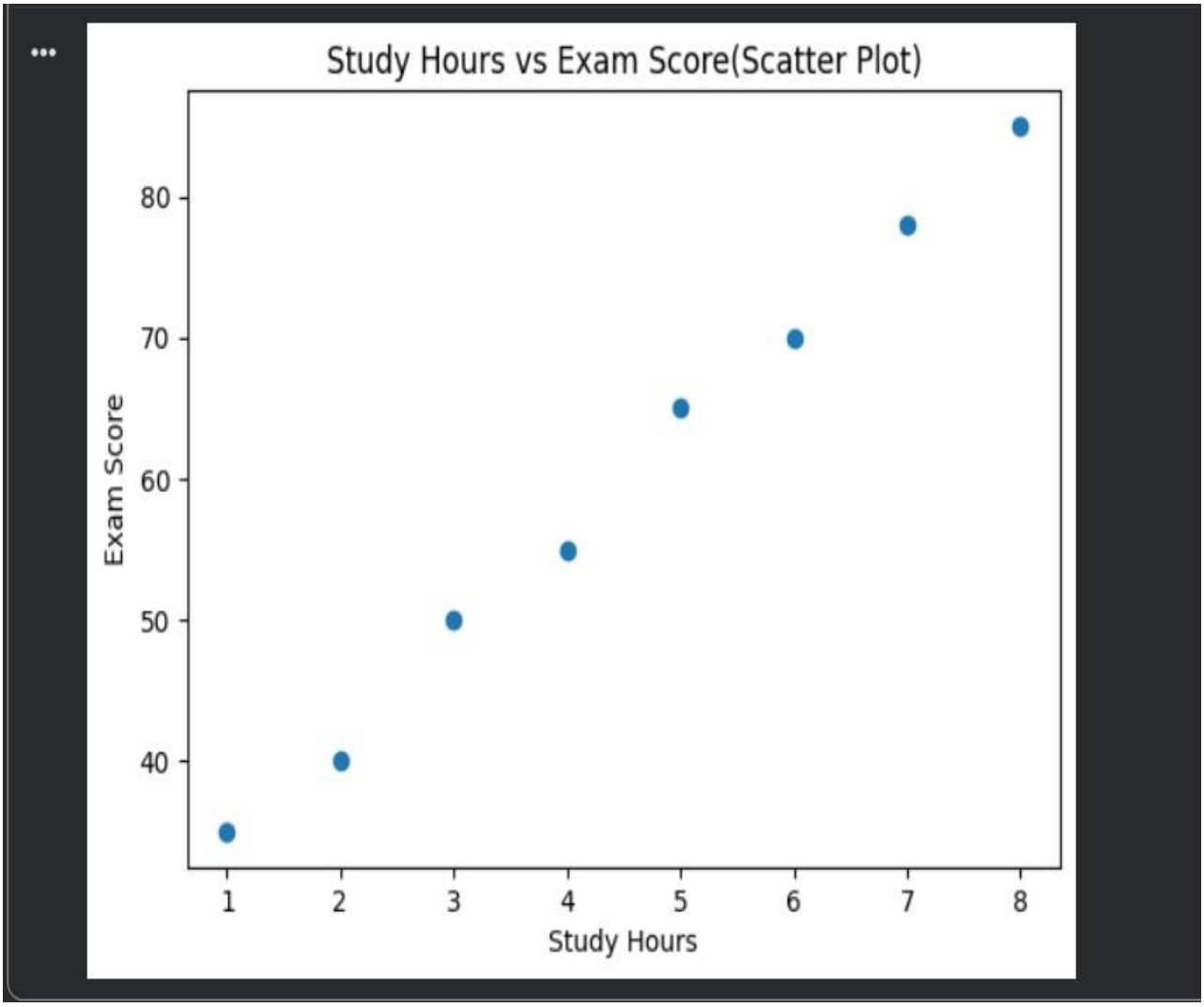
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data={'StudyHours':[1,2,3,4,5,6,7,8],
      'ExamScore':[35,40,50,55,65,70,78,85]}
df=pd.DataFrame(data)
print(df)
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
   StudyHours  ExamScore
0           1         35
1           2         40
2           3         50
3           4         55
4           5         65
5           6         70
6           7         78
7           8         85
```

1.Scatter Plot

```
plt.scatter(df['StudyHours'],df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score(Scatter Plot)")
plt.show()
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

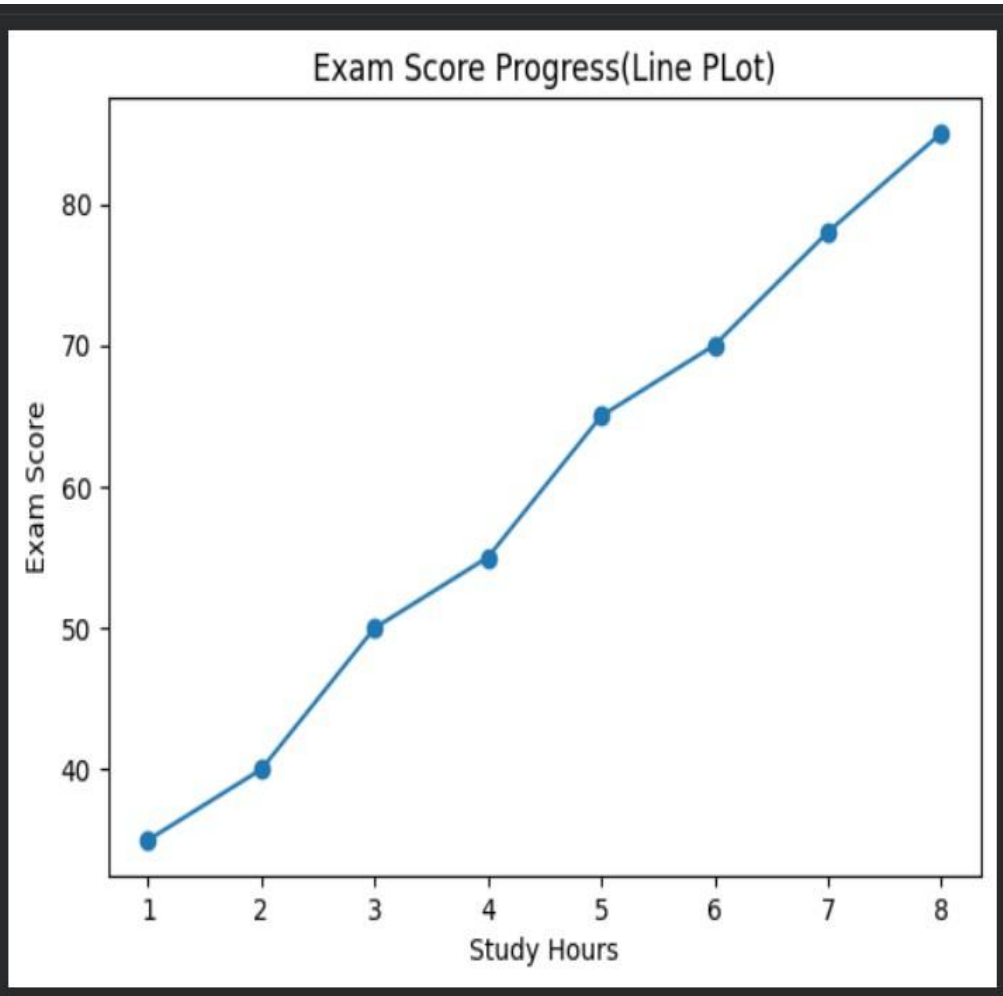
Output:



2.Line Plot

```
plt.plot(df['StudyHours'],df['ExamScore'],marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress(Line PLot)")
plt.show()
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

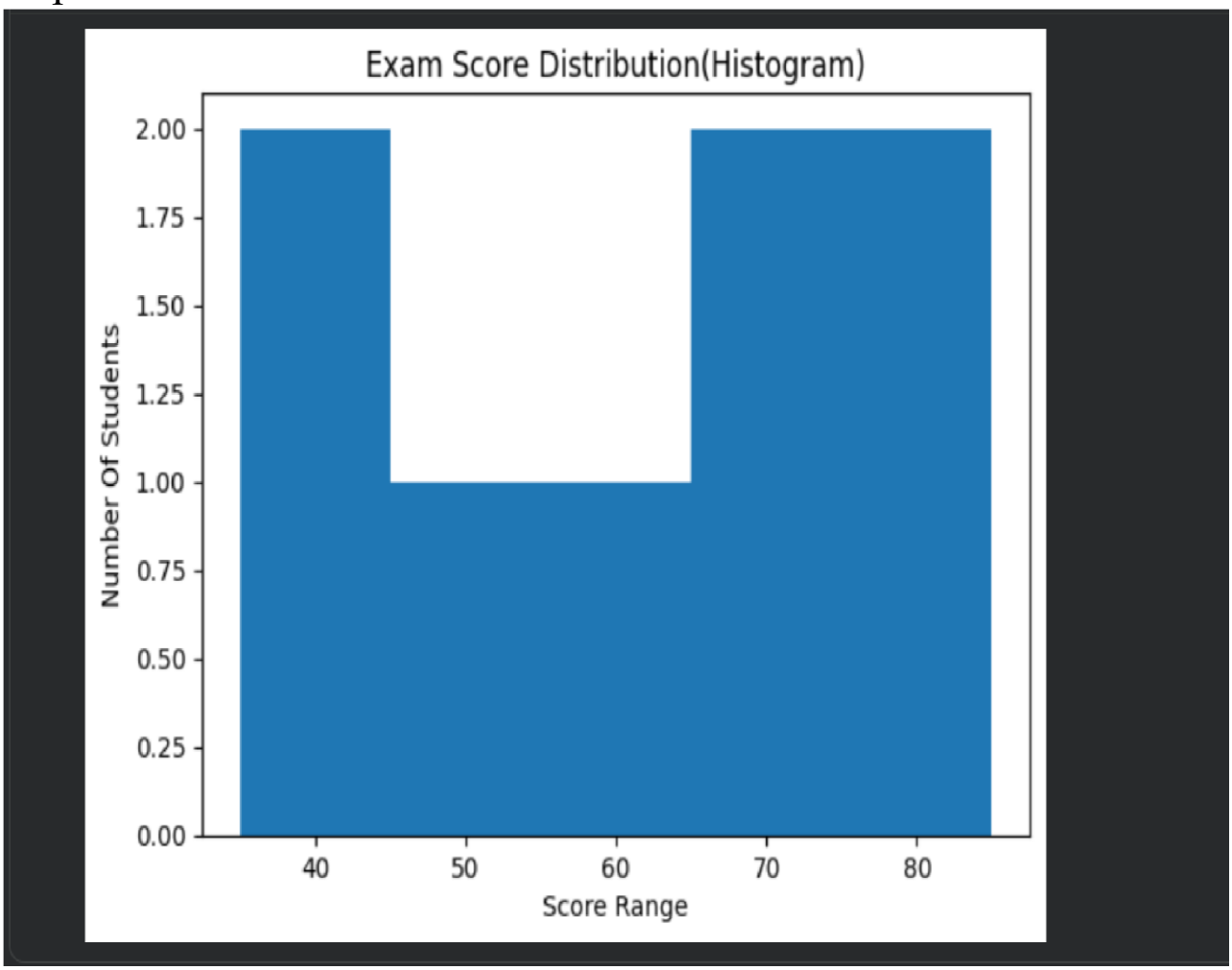
Output:



3.Histogram

```
plt.hist(df['ExamScore'],bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number Of Students")
plt.title("Exam Score Distribution(Histogram)")
plt.show()
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

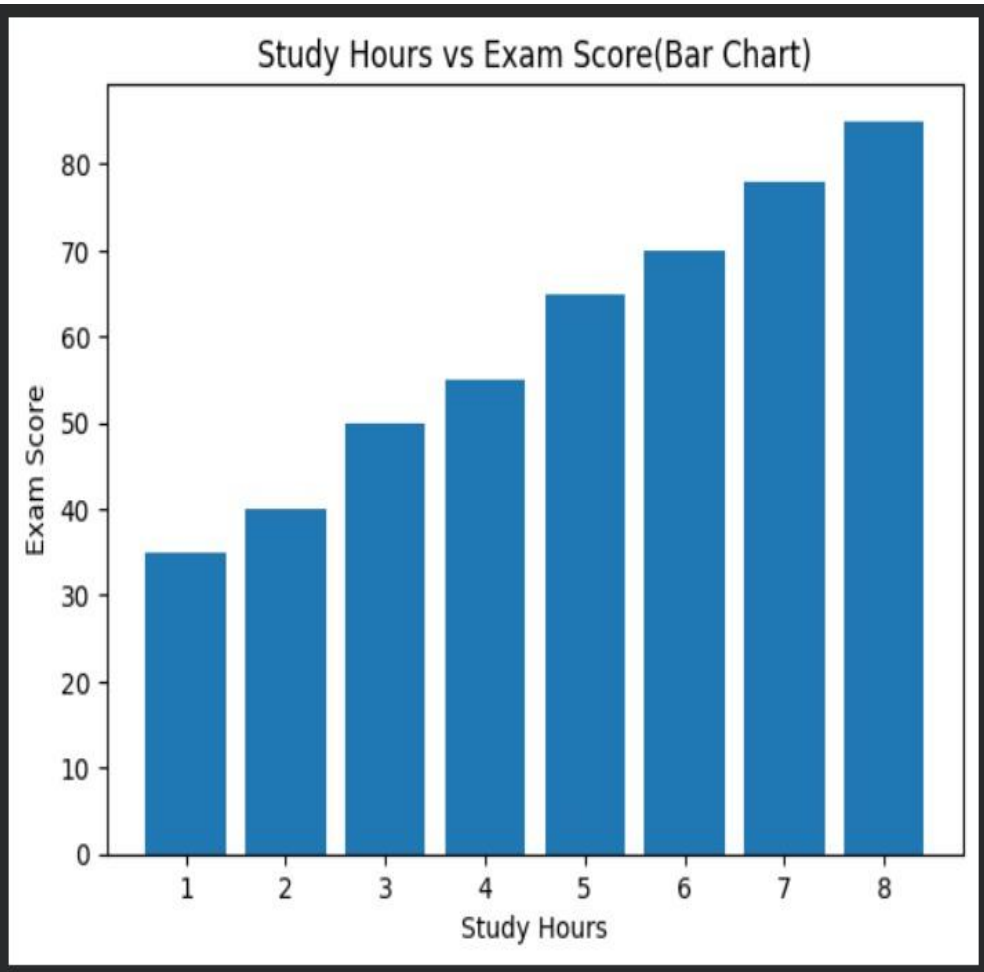
Output:



4.Bar Chart

```
plt.bar(df['StudyHours'],df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score(Bar Chart)")
plt.show()
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

Output:

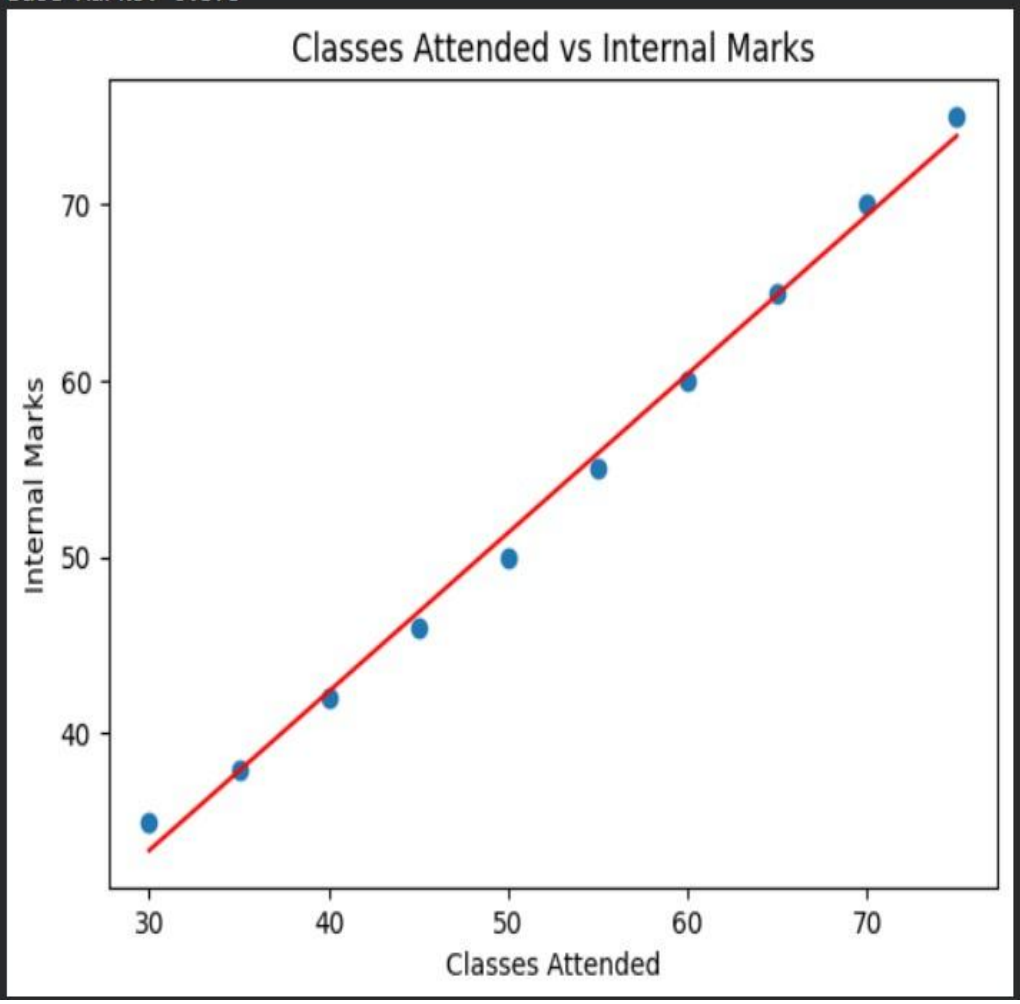


1.4) Scikit-learn (Machine Learning)

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
data = {
    'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
X = df[['Classes_Attended']]
y = df['Internal_Marks']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)
plt.scatter(X, y)
plt.plot(X, model.predict(X), color='red')
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

Output:

```
  Student  Classes_Attended  Internal_Marks
0      S1                30              35
1      S2                35              38
2      S3                40              42
3      S4                45              46
4      S5                50              50
5      S6                55              55
6      S7                60              60
7      S8                65              65
8      S9                70              70
9     S10                75              75
MSE: 0.578125
Marks per Class: 0.9
Base Marks: 6.375
```



2.Perform Data exploration and preprocessing in Python

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
data={'Country':['India','USA','India','USA','UK','India'],
     'Age':[22,25,np.nan,30,28,35],
     'Salary':[40000,60000,50000,np.nan,72000,58000],
     'Purchased':['No','Yes','Yes','No','Yes','Yes']}
df=pd.DataFrame(data)
print("---ORIGINAL RAW DATA---")
print(df)
print("\n")
print("---MISSING VALUES COUNT---")
print(df.isnull().sum())
print("\n")
df['Age']=df['Age'].fillna(df['Age'].mean())
df['Salary']=df['Salary'].fillna(df['Salary'].mean())
print("---DATA AFTER CLEANING---")
print(df)
print("\n")
df_encoded=pd.get_dummies(df,columns=['Country'])
df_encoded['Purchased']=df_encoded['Purchased'].map({'Yes':1,'No':0})
print("---DATA AFTER ENCODING (All Numbers NOW)---")
print(df_encoded)
print("\n")
X=df_encoded.drop('Purchased',axis=1)
y=df_encoded['Purchased']
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
print("---FINAL PROCESSED DATA (Ready For All)")
print(pd.DataFrame(X_scaled,columns=X.columns).head())
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

Output:

```
•••  ---ORIGINAL RAW DATA---
       Country   Age   Salary Purchased
     0   India  22.0  40000.0        No
     1     USA  25.0  60000.0       Yes
     2   India   NaN  50000.0       Yes
     3     USA  30.0      NaN        No
     4      UK  28.0  72000.0       Yes
     5   India  35.0  58000.0       Yes


     ---MISSING VALUES COUNT---
     Country      0
     Age          1
     Salary       1
     Purchased    0
     dtype: int64


     ---DATA AFTER CLEANING---
       Country   Age   Salary Purchased
     0   India  22.0  40000.0        No
     1     USA  25.0  60000.0       Yes
     2   India  28.0  50000.0       Yes
     3     USA  30.0  56000.0        No
     4      UK  28.0  72000.0       Yes
     5   India  35.0  58000.0       Yes
```

```
     ---DATA AFTER ENCODING (All Numbers NOW)---
         Age   Salary  Purchased  Country_India  Country_UK  Country_USA
     0  22.0  40000.0          0           True       False        False
     1  25.0  60000.0          1          False       False         True
     2  28.0  50000.0          1           True       False        False
     3  30.0  56000.0          0          False       False         True
     4  28.0  72000.0          1          False        True        False
     5  35.0  58000.0          1           True       False        False


     ---FINAL PROCESSED DATA (Ready For All)
             Age    Salary  Country_India  Country_UK  Country_USA
     0 -1.484615 -1.644453            1.0   -0.447214    -0.707107
     1 -0.742307  0.411113           -1.0   -0.447214     1.414214
     2  0.000000 -0.616670            1.0   -0.447214    -0.707107
     3  0.494872  0.000000           -1.0   -0.447214     1.414214
     4  0.000000  1.644453           -1.0    2.236068    -0.707107
```

3.Implement regularised Linear regression

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
diabetes = datasets.load_diabetes()
x_train, x_test, y_train, y_test = train_test_split(
    diabetes.data,
    diabetes.target,
    test_size=0.2,
    random_state=42
)
lr = LinearRegression().fit(x_train, y_train)
ridge = Ridge(alpha=1.0).fit(x_train, y_train)
lasso = Lasso(alpha=0.1).fit(x_train, y_train)
print(f"Linear Regression MSE: {mean_squared_error(y_test, lr.predict(x_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test, ridge.predict(x_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test, lasso.predict(x_test)):.2f}")
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 2798.19
```

4.Implement Naive Bayes classifier for dataset stored as CSV file.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
data = {
    'StudyHours': [3, 2, 8, 1, 6, 3, 7, 5],
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Pass']
}
df = pd.DataFrame(data)
X = df[['StudyHours', 'Attendance']]
y = df['Result']
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
model = GaussianNB()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Results:", y_pred)
print("Actual Results:", list(y_test.values))
print("Accuracy:", accuracy * 100, "%")
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

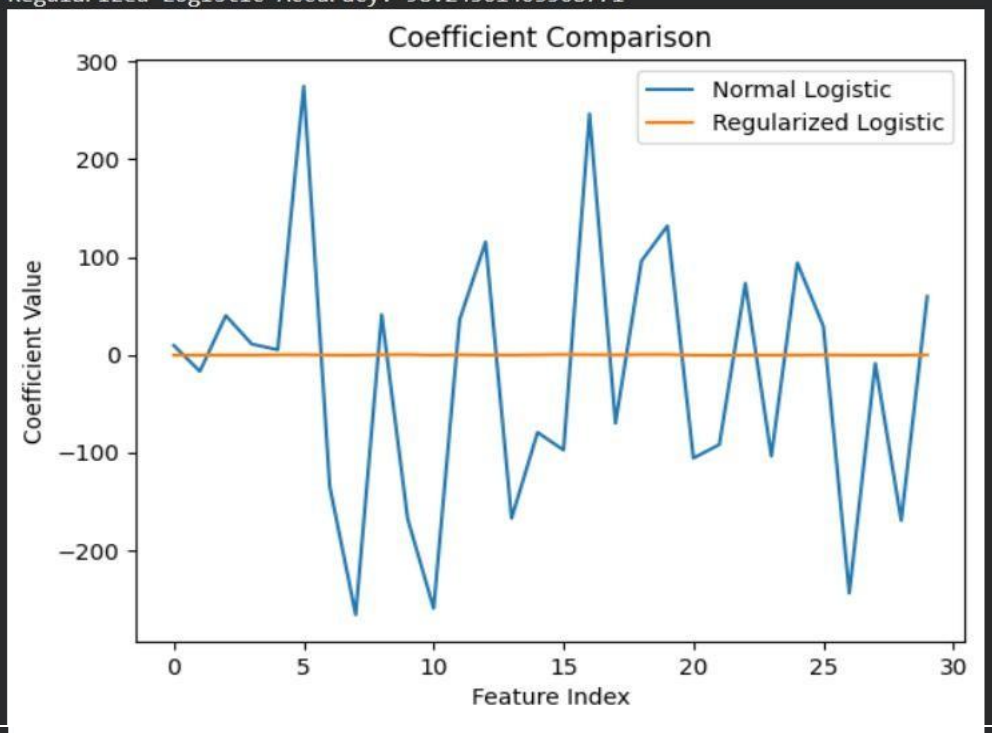
```
Predicted Results: ['Pass' 'Pass']
Actual Results: ['Fail', 'Fail']
Accuracy: 0.0 %
```

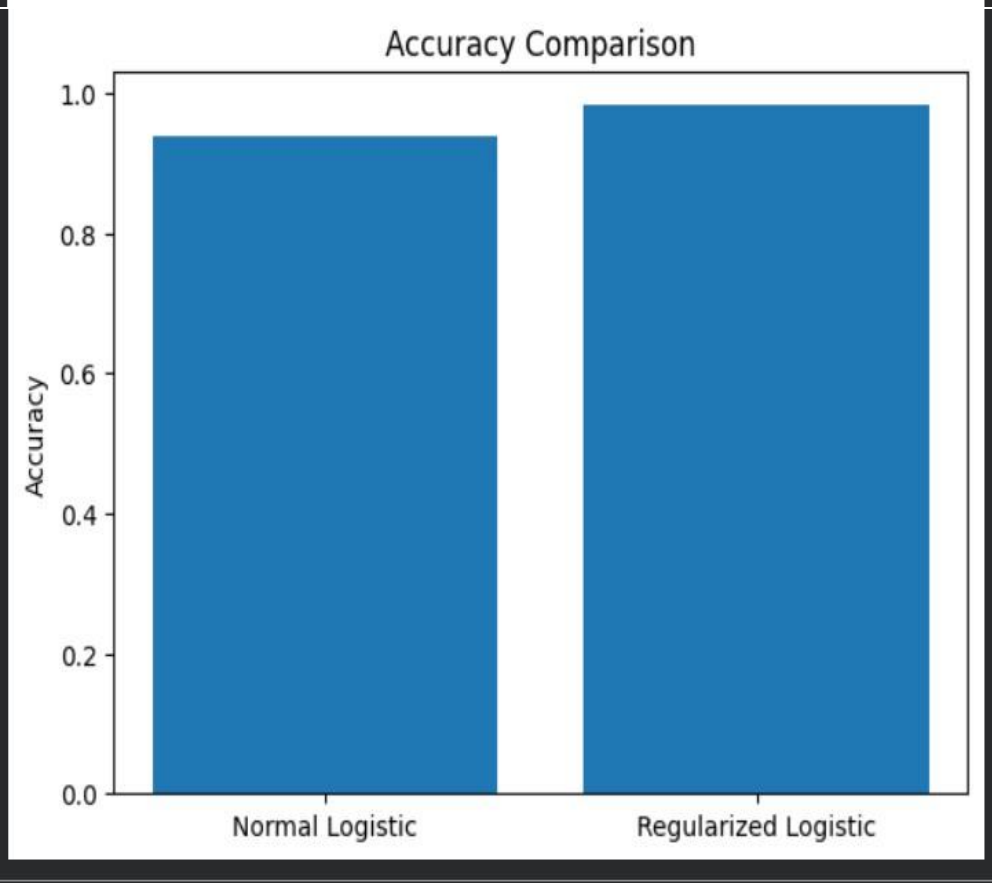
5.Implement regularized logistic regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
normal_model = LogisticRegression(penalty=None, solver='lbfgs', max_iter=1000)
normal_model.fit(X_train_scaled, y_train)
normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='lbfgs', max_iter=1000)
regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)
plt.figure()
plt.plot(normal_model.coef_.flatten(), label="Normal Logistic")
plt.plot(regularized_model.coef_.flatten(), label="Regularized Logistic")
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]
plt.figure()
plt.bar(models, accuracies)
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

Output:

```
Normal Logistic Accuracy: 93.85964912280701
Regularized Logistic Accuracy: 98.24561403508771
```





6.Build models using different Ensembling techniques

```python
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
).fit(X_train, y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

7.Build models using Decision trees

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
dt_clf = DecisionTreeClassifier(max_depth=3)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.        0.        0.39426873 0.60573127]
```

8. Build model using SVM with different kernels

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data=load_iris()
X=data.data
y=data.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
model_linear=SVC(kernel='linear')
model_linear.fit(X_train,y_train)
pred_linear=model_linear.predict(X_test)
acc_linear=accuracy_score(y_test,pred_linear)
print("Linear Kernel Accuracy:  ",acc_linear)
model_poly=SVC(kernel='poly',degree=3)
model_poly.fit(X_train,y_train)
pred_poly=model_poly.predict(X_test)
acc_poly=accuracy_score(y_test,pred_poly)
print("Polynomial Kernel Accuracy:",acc_poly)
model_rbf=SVC(kernel='rbf')
model_rbf.fit(X_train,y_train)
pred_rbf=model_rbf.predict(X_test)
acc_rbf=accuracy_score(y_test,pred_rbf)
print("RBF Kernel Accuracy:",acc_rbf)
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

```
Linear Kernel Accuracy:   0.9666666666666667
Polynomial Kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 0.9666666666666667
```

9.Build model using SVM with different kernels

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
data = {
"battery_power":[842,1021,563,615,1821,1859,1954,1445,509,700,1200,1600],
"ram":[2549,2631,2603,2769,1411,1067,700,1099,513,1500,2100,2800],
"px_height":[20,905,1263,1216,1208,1004,512,386,1137,500,800,900],
"px_width":[756,1988,1716,1786,1212,1654,1149,836,1224,900,1400,1700],
"price_range":[1,2,2,2,1,1,0,0,0,1,2,3]
}
df = pd.DataFrame(data)
X = df.drop("price_range",axis=1)
y = df["price_range"]
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.25,random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train,y_train)
y_pred = knn.predict(X_test)
print("Accuracy:",accuracy_score(y_test,y_pred))
print("Confusion Matrix:\n",confusion_matrix(y_test,y_pred))
print("Classification Report:\n",classification_report(y_test,y_pred,zero_division=0))
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

Output:

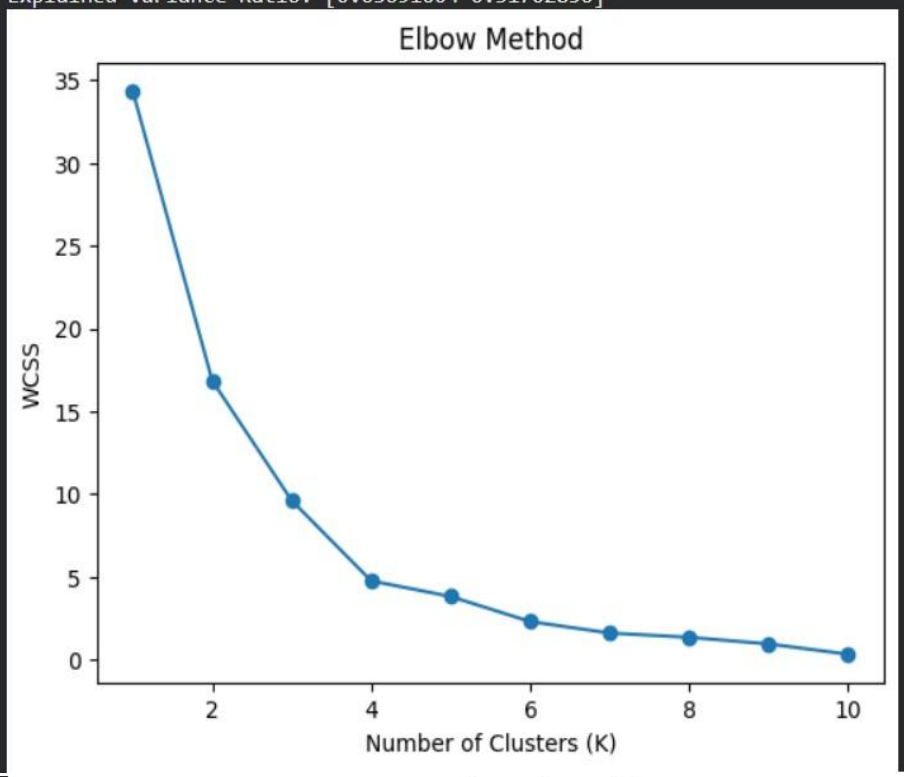
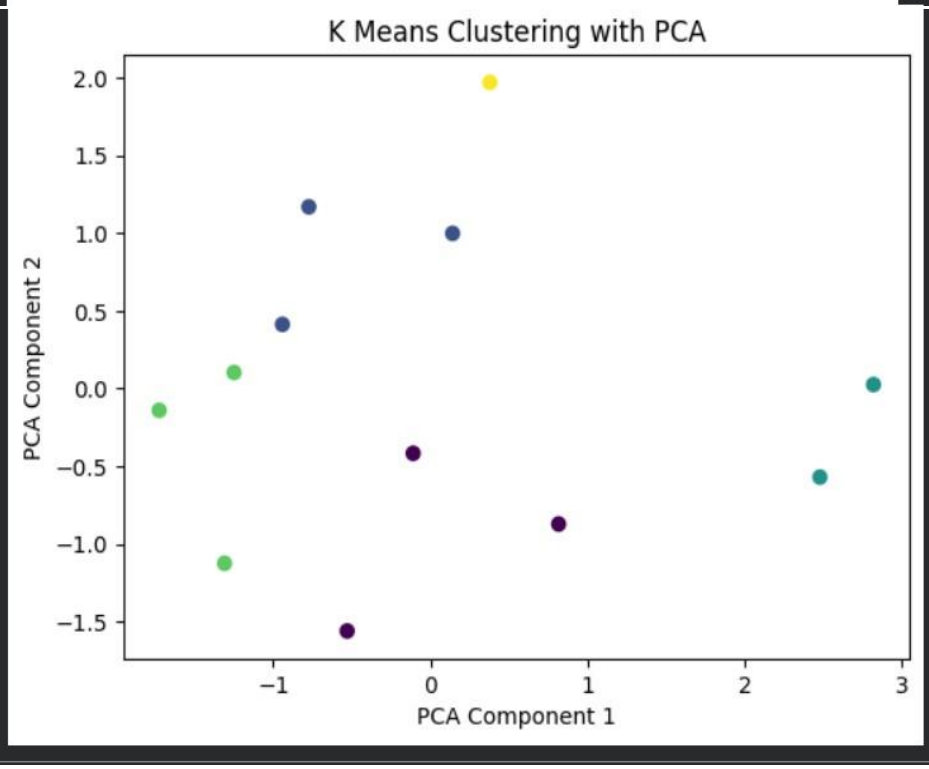
```
Accuracy: 0.0
Confusion Matrix:
 [[0 0 0]
 [2 0 0]
 [0 1 0]]
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00       0.0
           1       0.00      0.00      0.00       2.0
           2       0.00      0.00      0.00       1.0

    accuracy                           0.00       3.0
   macro avg       0.00      0.00      0.00       3.0
weighted avg       0.00      0.00      0.00       3.0
```

10. Build model to perform Clustering using K-means after applying PCA and determining the value of K using Elbow Method

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data = {
"CustomerID":[1,2,3,4,5,6,7,8,9,10,11,12],
"Age":[19,21,20,23,31,22,35,23,64,30,67,35],
"Annual Income (k$)":[15,15,16,16,17,17,18,18,19,19,20,20],
"Spending Score (1-100)":[39,81,6,77,40,76,6,94,3,72,14,99]}
data = pd.DataFrame(data)
X = data[['Age','Annual Income (k$)','Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0)
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters)
plt.title("K Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
#SK.MOHAMMED IRFAN REG.NO:11249A342
```

Output:

```
Explained Variance Ratio: [0.63691004 0.31702856]
```